

Requirements:

```
fastapi==0.95.2
uvicorn[standard]==0.22.0
opencv-python==4.8.0
numpy==1.26.0
pydantic==1.10.12
ultralytics==8.0.134      # YOLOv8 (or use yolov5 if preferred)
facenet-pytorch==2.5.2
deep_sort_realtime==1.5.5
mediapipe==0.10.1
torch==2.2.0
torchvision==0.15.2
aiofiles==23.1.0
python-multipart==0.0.6
typing_extensions==4.9.0
```

CODE

```
# app.py
import io
import time
import asyncio
from typing import List, Dict, Any, Optional
from fastapi import FastAPI, File, UploadFile, WebSocket, WebSocketDisconnect
from pydantic import BaseModel
import numpy as np
import cv2
from PIL import Image
import base64
import uuid

# Detection & ML libs
from ultralytics import YOLO
import torch
from facenet_pytorch import InceptionResnetV1, MTCNN
from deep_sort_realtime.deepsort_tracker import DeepSort
import mediapipe as mp

app = FastAPI(title="Unified Human Detection Backend")

# -----
# CONFIG / LOAD MODELS
# -----
```

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
```

```
# 1) YOLO model for person/body/head detection. Use a model with appropriate classes or  
custom trained.
```

```
yolo_model = YOLO("yolov8n.pt") # small; swap to custom trained for head/upper-body splits  
yolo_model.to(DEVICE)
```

```
# 2) Face detector + face embedder
```

```
mtcnn = MTCNN(keep_all=True, device=DEVICE)
```

```
face_embedder = InceptionResnetV1(pretrained='vggface2').eval().to(DEVICE)
```

```
# 3) Tracker (DeepSort)
```

```
tracker = DeepSort(max_age=30, n_init=3, nms_max_overlap=1.0, max_cosine_distance=0.4)
```

```
# 4) Mediapipe for pose
```

```
mp_pose = mp.solutions.pose
```

```
pose_processor = mp_pose.Pose(static_image_mode=False, min_detection_confidence=0.5)
```

```
# Simple in-memory face DB: id -> {"name": str, "embedding": np.array}
```

```
FACE_DB: Dict[str, Dict[str, Any]] = {}
```

```
# For stationary object detection
```

```
bg_subtractor = cv2.createBackgroundSubtractorMOG2(history=500, detectShadows=True)
```

```
stationary_objects: Dict[str, Dict[str, Any]] = {} # id -> {first_seen_ts, last_seen_ts, bbox,  
stationary_since}
```

```
# Persistence or long-term DB omitted (replace with Redis/DB in prod)
```

```
# -----
```

```
# Utilities
```

```
# -----
```

```
def read_imagefile_to_cv2(data: bytes):
```

```
    img = Image.open(io.BytesIO(data)).convert("RGB")
```

```
    arr = np.array(img)[::-1].copy() # RGB->BGR
```

```
    return arr
```

```
def encode_image_to_base64(img_bgr):
```

```
    _, buffer = cv2.imencode('.jpg', img_bgr)
```

```
    return base64.b64encode(buffer).decode('utf-8')
```

```
def bbox_area(box):
```

```
    x1,y1,x2,y2 = box
```

```
    return max(0, x2-x1) * max(0, y2-y1)
```

```

# -----
# Face handling
# -----
def register_face_from_image(img_bgr, name: str) -> str:
    # Detect face(s), take first, compute embedding, store in DB
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    pil = Image.fromarray(img_rgb)
    boxes, _ = mtcnn.detect(pil)
    if boxes is None or len(boxes) == 0:
        raise ValueError("No face found")
    box = boxes[0]
    x1,y1,x2,y2 = [int(v) for v in box]
    face = img_rgb[y1:y2, x1:x2]
    face_pil = Image.fromarray(face)
    face_tensor = mtcnn.extract(face_pil, None).to(DEVICE)
    emb = face_embedder(face_tensor.unsqueeze(0)).detach().cpu().numpy()[0]
    face_id = str(uuid.uuid4())
    FACE_DB[face_id] = {"name": name, "embedding": emb.tolist(), "registered_at": time.time()}
    return face_id

def compute_face_embeddings_from_pil(pil_img):
    # pil_img is PIL RGB
    boxes, probs = mtcnn.detect(pil_img)
    if boxes is None: return []
    embeddings = []
    faces_tensors = []
    for box in boxes:
        x1,y1,x2,y2 = [int(v) for v in box]
        face = pil_img.crop((x1,y1,x2,y2))
        face_tensor = mtcnn.extract(face, None).to(DEVICE)
        faces_tensors.append(face_tensor)
    if not faces_tensors:
        return []
    faces_tensor = torch.stack(faces_tensors).to(DEVICE)
    with torch.no_grad():
        emb = face_embedder(faces_tensor).cpu().numpy()
    return emb, boxes

def match_embedding_to_db(embedding, threshold=0.8):
    # simple cosine similarity
    if not FACE_DB:
        return None, 0.0
    emb = np.array(embedding)

```

```

best_id, best_sim = None, -1.0
for fid, rec in FACE_DB.items():
    dbemb = np.array(rec["embedding"])
    # cosine
    sim = np.dot(emb, dbemb) / (np.linalg.norm(emb) * np.linalg.norm(dbemb) + 1e-8)
    if sim > best_sim:
        best_sim = sim
        best_id = fid
if best_sim >= threshold:
    return best_id, float(best_sim)
return None, float(best_sim)

# -----
# Main detection pipeline for single frame
# -----
class DetectionResult(BaseModel):
    presence: bool
    persons: List[Dict[str, Any]]
    person_count: int
    crowd_density: float
    faces: List[Dict[str, Any]]
    skeletons: List[Dict[str, Any]]
    movement_events: List[Dict[str, Any]]
    stationary_objects: List[Dict[str, Any]]

@app.post("/process_frame", response_model=DetectionResult)
async def process_frame(file: UploadFile = File(...), camera_id: Optional[str] = None):
    """
    POST an image file (frame) to analyze. Returns detections and analytics.
    """
    content = await file.read()
    frame = read_imagefile_to_cv2(content)
    h, w = frame.shape[:2]

    # 1) Run YOLO detection (person + optionally head/body classes)
    # Use yolo predict: returns boxes, scores, cls
    # NOTE: ultralytics returns normalized coordinates depending on version; adjust accordingly.
    results = yolo_model.predict(source=frame, conf=0.3, device=DEVICE, verbose=False)
    res = results[0] # single image
    dets = []
    for r in res.boxes:
        xxyy = r.xyxy[0].cpu().numpy() # x1,y1,x2,y2
        conf = float(r.conf[0].cpu().numpy())
        cls = int(r.cls[0].cpu().numpy())

```

```

dets.append({"xyxy": xyxy.tolist(), "conf": conf, "class": cls})

# Filter person detections (YOLO class 0 usually in COCO)
persons = [d for d in dets if d["class"] == 0]
person_count = len(persons)
presence = person_count > 0

# 2) Convert to tracker input format and update tracker to get track ids
detections_for_tracker = []
for p in persons:
    x1,y1,x2,y2 = [int(x) for x in p["xyxy"]]
    bbox = [x1, y1, x2-x1, y2-y1] # xywh
    detections_for_tracker.append({'bbox': bbox, 'confidence': p['conf'], 'class': 'person'})

tracks = tracker.update_tracks(detections_for_tracker, frame=frame)
persons_out = []
centroids = []
movement_events = []
skeletons_out = []

# 3) Face detection + recognition (MTCNN)
pil_rgb = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
face_embeddings_and_boxes = compute_face_embeddings_from_pil(pil_rgb)
faces_out = []
if face_embeddings_and_boxes:
    embs, boxes = face_embeddings_and_boxes
    for emb, box in zip(embs, boxes):
        fid, sim = match_embedding_to_db(emb, threshold=0.75)
        x1,y1,x2,y2 = [int(v) for v in box]
        faces_out.append({"bbox": [x1,y1,x2,y2], "match_id": fid, "similarity": sim})

# 4) Skeleton (pose) by cropping full frame (mediapipe)
mp_frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
pose_results = pose_processor.process(mp_frame_rgb)
if pose_results.pose_landmarks:
    # convert mediapipe landmarks
    lm = pose_results.pose_landmarks.landmark
    keypoints = [{"x": lm.x, "y": lm.y, "z": lm.z, "visibility": lm.visibility} for lm in lm]
    skeletons_out.append({"keypoints": keypoints})

# 5) For each track, compute centroid and direction, annotate person_out
for t in tracks:
    if not t.is_confirmed():
        continue

```

```

tid = t.track_id
l, t_, w_, h_ = t.to_ltrwh()
bbox = [int(l), int(t_), int(l+w_), int(t_+h_)]
cx = int((bbox[0]+bbox[2]) / 2)
cy = int((bbox[1]+bbox[3]) / 2)
centroids.append((cx,cy))

```

track metadata for direction detection: store last positions on tracker object (we'll attach to tracker)

```

if not hasattr(t, "history"):
    t.history = []
t.history.append((cx,cy, time.time()))
# keep limited history
t.history = t.history[-10:]

```

```

direction = "unknown"
if len(t.history) >= 2:
    dx = t.history[-1][0] - t.history[0][0]
    dy = t.history[-1][1] - t.history[0][1]
    if abs(dx) > abs(dy):
        direction = "right" if dx > 0 else "left"
    else:
        direction = "down" if dy > 0 else "up"
# event detection
movement_events.append({"track_id": tid, "direction": direction, "dx": dx, "dy": dy})

```

```

persons_out.append({"track_id": tid, "bbox": bbox, "centroid": [cx,cy], "direction": direction})

```

6) Crowd density: persons per 10000 px² (simple)

```

total_area = h * w
density = person_count / (total_area / 10000.0) if total_area > 0 else 0.0

```

7) Movement detection & stationary object detection using background subtractor

```

fgmask = bg_subtractor.apply(frame)
# morphological cleaning
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_OPEN, kernel)
contours, _ = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
stationary_out = []
for cnt in contours:
    area = cv2.contourArea(cnt)
    if area < 500: continue
    x,y,w_,h_ = cv2.boundingRect(cnt)

```

```

bbox = [x,y,x+w_,y+h_]
key = f"{x}_{y}_{w}_{h}"
ts = time.time()
if key not in stationary_objects:
    stationary_objects[key] = {"first_seen": ts, "last_seen": ts, "bbox": bbox,
"stationary_since": None}
else:
    stationary_objects[key]["last_seen"] = ts

# if object persists beyond threshold, mark as stationary
persist = stationary_objects[key]["last_seen"] - stationary_objects[key]["first_seen"]
if persist > 10: # seconds threshold -> tune as needed
    stationary_objects[key]["stationary_since"] = stationary_objects[key]["first_seen"]
    stationary_out.append({"bbox": bbox, "first_seen": stationary_objects[key]["first_seen"],
"persist_sec": persist})

# Build response
out = {
    "presence": presence,
    "persons": persons_out,
    "person_count": person_count,
    "crowd_density": float(density),
    "faces": faces_out,
    "skeletons": skeletons_out,
    "movement_events": movement_events,
    "stationary_objects": stationary_out
}
return out

# -----
# Utility endpoints for face DB
# -----
class FaceRegisterReq(BaseModel):
    name: str

@app.post("/register_face")
async def register_face(name: str, file: UploadFile = File(...)):
    content = await file.read()
    frame = read_imagefile_to_cv2(content)
    try:
        fid = register_face_from_image(frame, name)
        return {"face_id": fid, "name": name}
    except ValueError as e:
        return {"error": str(e)}

```

```

@app.get("/faces")
async def list_faces():
    return FACE_DB

# -----
# Websocket simple stream example (receives JPEG frames)
# -----
class StreamingConnection:
    def __init__(self):
        self.connections: List[WebSocket] = []

streaming = StreamingConnection()

@app.websocket("/ws/stream/{camera_id}")
async def ws_stream(websocket: WebSocket, camera_id: str):
    await websocket.accept()
    streaming.connections.append(websocket)
    try:
        while True:
            # expect client to send base64 JPEG frames or binary JPEG
            msg = await websocket.receive_bytes()
            frame = read_imagefile_to_cv2(msg)
            # reuse process_frame logic but call sync function via threadpool if heavy
            loop = asyncio.get_event_loop()
            # use same pipeline synchronously but in threadpool if needed: simplified here
            # For performance, do detection asynchronously or in a worker
            results = await process_frame_file_bytes(msg, camera_id)
            await websocket.send_json(results)
    except WebSocketDisconnect:
        streaming.connections.remove(websocket)
    except Exception as e:
        try:
            await websocket.send_json({"error": str(e)})
        except:
            pass

# helper wrapper to call the same pipeline using bytes
async def process_frame_file_bytes(frame_bytes: bytes, camera_id: Optional[str] = None):
    # reuse above code but duplicate minimal necessary to avoid FastAPI injection
    frame = read_imagefile_to_cv2(frame_bytes)
    # run the same steps: YOLO -> tracker -> faces -> pose -> bg subtractor, etc.
    # for brevity call the /process_frame endpoint logic by directly calling function (works because
    it's pure)

```



```
class DummyUpload:
    async def read(self, non):
        return frame_bytes
    return await process_frame(DummyUpload(), camera_id)

# -----
# Run server
# -----
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app:app", host="0.0.0.0", port=8000, reload=True)
```